

1895. Largest Magic Square

Solved 

Medium

 Topics

 Companies

 Hint

A $k \times k$ **magic square** is a $k \times k$ grid filled with integers such that every row sum, every column sum, and both diagonal sums are **all equal**. The integers in the magic square **do not have to be distinct**. Every 1×1 grid is trivially a **magic square**.

Given an $m \times n$ integer `grid`, return the **size** (i.e., the side length k) of the **largest magic square** that can be found within this grid.

Example 1:

7	1	4	5	6
2	5	1	6	4
1	5	4	3	2
1	2	7	3	4

Input: `grid = [[7,1,4,5,6],[2,5,1,6,4],[1,5,4,3,2],[1,2,7,3,4]]`

Output: 3

Explanation: The largest magic square has a size of 3.

Every row sum, column sum, and diagonal sum of this magic square is equal to 12.

- Row sums: $5+1+6 = 5+4+3 = 2+7+3 = 12$
- Column sums: $5+5+2 = 1+4+7 = 6+3+3 = 12$
- Diagonal sums: $5+4+3 = 6+4+2 = 12$

Example 2:

5	1	3	1
9	3	3	1
1	3	3	8

Input: `grid = [[5,1,3,1],[9,3,3,1],[1,3,3,8]]`

Output: 2

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 50`
- `1 <= grid[i][j] <= 106`

Python:

class Solution:

```
def largestMagicSquare(self, grid: List[List[int]]) -> int:  
    m, n = len(grid), len(grid[0])  
    res = 1
```

```

def isValid(i, j, k):
    s = None
    for x in range(i, i + k):
        row = sum(grid[x][j:j + k])
        if s is None: s = row
        elif s != row: return False

    for y in range(j, j + k):
        if sum(grid[x][y] for x in range(i, i + k)) != s:
            return False

    if sum(grid[i + d][j + d] for d in range(k)) != s:
        return False

    if sum(grid[i + d][j + k - 1 - d] for d in range(k)) != s:
        return False

    return True

for k in range(2, min(m, n) + 1):
    for i in range(m - k + 1):
        for j in range(n - k + 1):
            if isValid(i, j, k):
                res = k
return res

```

JavaScript:

```

var largestMagicSquare = function(grid) {
    const m = grid.length, n = grid[0].length;
    let res = 1;

    const isValid = (i, j, k) => {
        let sum = 0;

        for (let x = i; x < i + k; x++) {
            let s = 0;
            for (let y = j; y < j + k; y++) s += grid[x][y];
            if (x === i) sum = s;
            else if (sum !== s) return false;
        }

        for (let y = j; y < j + k; y++) {
            let s = 0;
            for (let x = i; x < i + k; x++) s += grid[x][y];

```

```

        if (sum !== s) return false;
    }

    let s = 0;
    for (let d = 0; d < k; d++) s += grid[i + d][j + d];
    if (sum !== s) return false;

    s = 0;
    for (let d = 0; d < k; d++) s += grid[i + d][j + k - 1 - d];
    return sum === s;
};

for (let k = 2; k <= Math.min(m, n); k++) {
    for (let i = 0; i + k <= m; i++) {
        for (let j = 0; j + k <= n; j++) {
            if (isValid(i, j, k)) res = k;
        }
    }
}
return res;
};

```

Java:

```

class Solution {
    public int largestMagicSquare(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int side = Math.min(m,n);

        while(side > 0){
            for(int i=0;i<m;i++){
                for(int j=0;j<n;j++){
                    if(i+side<=m && j+side<=n){
                        if(isValid(grid,m,n,i,j,side))return side;
                    }
                }
            }
            side--;
        }

        return 0;
    }

    private boolean isValid(int[][] grid, int m, int n, int i, int j, int side) {
        int sum = 0;

```

```

// rows
for (int x = i; x < i + side; x++) {
    int summ = 0;
    for (int y = j; y < j + side; y++) {
        summ += grid[x][y];
    }
    if (x == i) sum = summ;
    else if (sum != summ) return false;
}

// columns
for (int x = j; x < j + side; x++) {
    int summ = 0;
    for (int y = i; y < i + side; y++) {
        summ += grid[y][x];
    }
    if (sum != summ) return false;
}

// main diagonal
int summ = 0;
for (int k = 0; k < side; k++) {
    summ += grid[i + k][j + k];
}
if (sum != summ) return false;

// anti-diagonal
summ = 0;
for (int k = 0; k < side; k++) {
    summ += grid[i + k][j + side - 1 - k];
}
if (sum != summ) return false;

return true;
}
}

```